pandas I: load, look and filter a table of 30,229 orders

How do we open a file of 30,229 rows, look at it, and ask it the first questions?

AI and Data Science Professional | Session 7 of 24 | Module B: Python for data
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Load a CSV file into a pandas DataFrame and take a first look: shape, data types, missing values and describe



Select columns and filter rows with a Boolean mask, join two conditions with & and |, and write the same filter with query



Sort, count with value_counts, pick rows with loc and iloc, and save a data profile of the file

Your two hours: concepts first, then a pandas lab

0:00	10	Recall and the question of the day
0:10	45	Concepts: picture, plain words, one example by hand
0:55	10	Break
1:05	45	Lab: predict, run, read, change
1:50	10	Exit check, keep and next

Ideas with pictures first. Then you open the notebook. Then the checks and what you keep.

Three questions from last time

1. You wrote a function that gives a loyalty tier from spend. How did you test it?

2. In Excel you drag a formula down 5,000 rows. What does a for loop do instead?

3. What does the word return do inside a function?



Answer aloud first. Then we check together.

Answers to the three questions from Session 6

	Question	Short answer
	**You wrote a function that gives a loyalty tier from spend. How did you test it?**	Run it on examples whose answer you already know, including the values exactly on each tier boundary.
	**In Excel you drag a formula down 5,000 rows. What does a for loop do instead?**	It repeats the same steps once for each item in a list, so you write the step one time.
	**What does the word return do inside a function?**	It hands the result back to the code that called the function, so you can store it or print it.

Words we use today

	Word	What it means
	**DataFrame**	A table held in Python memory, with labelled rows and named columns. In Excel words: one tab of a workbook.
	**Series**	One column of a DataFrame. It has a name, a data type and the same index as the table.
	**Index**	The row labels down the left edge of a DataFrame. A fresh file is labelled 0, 1, 2 and so on.
	**Boolean mask (Boolean indexing)**	A column of True and False values, one per row, used to keep only the rows marked True.
	**Missing value (NaN)**	A cell with no value. pandas shows it as NaN, SQL as NULL, and Excel as a blank cell.

A pandas DataFrame is a table in memory, and its index labels the rows

orders: a DataFrame, the whole table

shape: (30,229, 7)

index ↓ | **column names** ↓

	order_id	order_date	branch_id	customer_id	payment_method	promo_week	order_value_ugx
0	O000001	2024-01-06	B01	C0495	Card	0	56000
1	O000002	2024-01-03	B01		Cash	0	80600
2	O000003	2024-01-07	B01		Cash	0	66200
3	O000004	2024-01-03	B01	C1815	Cash	0	56000
4	O000005	2024-01-01	B01	C0453	Cash	0	3400

Series: one column, branch_id

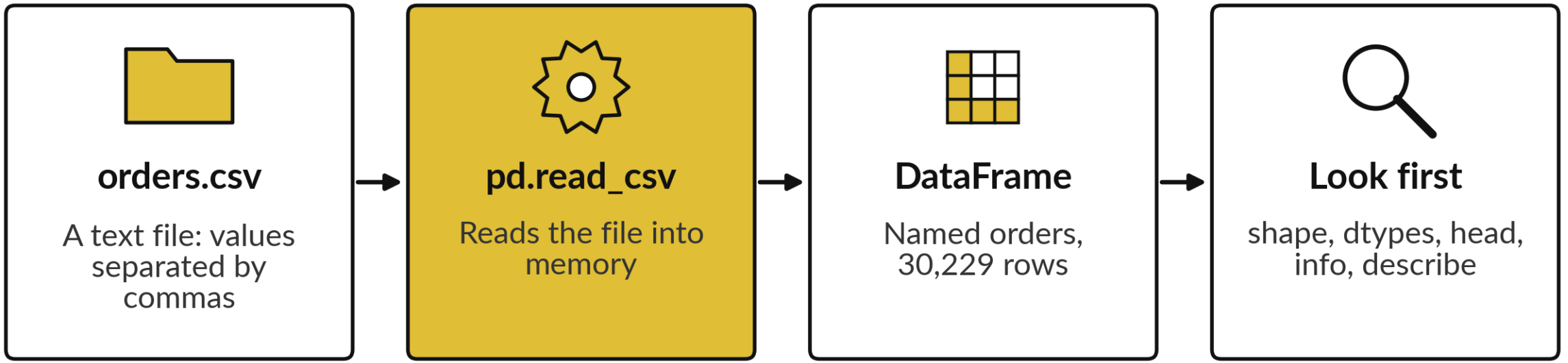
A Series is one column of a DataFrame. The shape is rows first, then columns.

The Excel word, the pandas word and what each means

	In Excel	In pandas	Plain meaning
	**A tab**	DataFrame	A table held in Python memory.
	**A column**	Series (a variable)	One column, with a name and a data type.
	**Row numbers**	Index	The label of each row.
	**AutoFilter**	Boolean indexing	Keep the rows where a test is True.
	**Sort A to Z**	sort_values	Order the rows by a column.
	**PivotTable count**	value_counts	Count each different value.

A row is called a record or an observation. A column is called a variable, or a feature in machine learning.

read_csv turns a file into a DataFrame, and then we look before we ask



CSV means comma-separated values. In Colab the file sits next to the notebook, so its name is enough. pd is the usual short name for the pandas library.

shape gives the size, and dtypes gives the data type of every column

```
orders = pd.read_csv('orders.csv')
orders.shape
```

```
(30229, 7)
```

30,229 records (rows) and 7 variables (columns). shape has no brackets after it.

```
orders.dtypes
```

```
order_id           object
order_date         object
branch_id          object
customer_id        object
payment_method     object
promo_week          int64
order_value_ugx     int64
dtype: object
```

object means text (a string; pandas 3 and later shows str). order_date is stored as text, so it cannot yet be sorted as a date.

int64 is a whole number. promo_week is a 0 or 1 flag.

A data type is how a column stores its values. Session 8 converts the dates.

info lists each column with how many values are filled, and customer_id is short

```
orders.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30229 entries, 0 to 30228
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   order_id         30229 non-null  object
 1   order_date       30229 non-null  object
 2   branch_id        30229 non-null  object
 3   customer_id      21058 non-null  object
 4   payment_method   30229 non-null  object
 5   promo_week       30229 non-null  int64
 6   order_value_ugx  30229 non-null  int64
dtypes: int64(2), object(5)
memory usage: 1.6+ MB
```

Non-Null Count: how many cells hold a value.

Only 21,058 of 30,229 orders have a customer_id. The rest are missing.

Python lists the data types it found.

info is a method: it has round brackets. Non-null means "not missing".

isna().sum() counts the missing values, and 30 percent of orders have no customer_id

```
orders.isna().sum()
```

```
order_id             0
order_date           0
branch_id            0
customer_id       9171
payment_method       0
promo_week           0
order_value_ugx      0
dtype: int64
```

A missing value (NaN) is a cell with no value. 9,171 orders have no customer_id.

isna() marks each empty cell True. sum() adds the Trues.

Ask what the blank means. Here it is a walk-in shopper.

Do not delete 30 percent of the orders.

describe summarises a numeric variable, and the mean is 2.05 times the median

```
orders['order_value_ugx'].describe().round().astype(int)
```

```
count      30229
mean       47491
std       123574
min          800
25%         9940
50%        23200
75%        47300
max      3104000
Name: order_value_ugx, dtype: int64
```

mean: the arithmetic mean, UGX 47,491

50% is the median: UGX 23,200. Half the orders are smaller.

max: UGX 3,104,000, one very large order

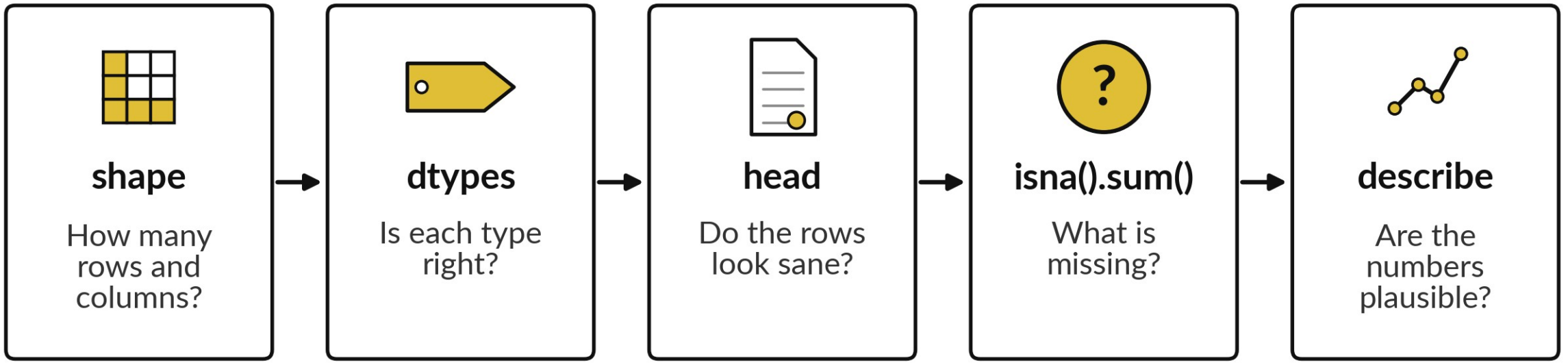
A few very large orders pull the mean above the median: the variable is right-skewed.

pandas data types and the variable types they usually store

	pandas dtype	Stores	Example in our files
	**object (shown as str in pandas 3 and later)**	Text (a string)	order_id, branch_id, order_date
	**int64**	Whole numbers	promo_week, order_value_ugx
	**float64**	Numbers with decimals, and NaN	monthly_income_ugx in loans.csv
	**bool**	True or False	A Boolean mask
	**datetime64**	Dates and times	order_date after Session 8

A whole-number column that has a missing value becomes float64, because NaN is a float.

The first look at any file: five calls before any question



This routine is the start of exploratory data analysis (EDA): getting to know a dataset before you answer questions with it.

One column name returns a Series, and a list of names returns a DataFrame

```
orders['branch_id'].head(3)
```

```
0    B01
1    B01
2    B01
Name: branch_id, dtype: object
```

A Series: one column. The last line gives its name and data type.

```
orders[['branch_id', 'order_value_ugx']].head(3)
```

```
  branch_id  order_value_ugx
0       B01            56000
1       B01            80600
2       B01            66200
```

A DataFrame: two columns. The inner brackets make the list.

Selecting columns is like hiding the other columns in Excel. head(3) keeps three rows.

A Boolean mask is a column of True and False that chooses the rows to keep

1. The rows (7 of them)

	order_id	branch_id
14056	O014057	B03
14057	O014058	B03
14058	O014059	B03
14059	O014060	B03
14060	O014061	B04
14061	O014062	B04
14062	O014063	B04

2. The mask (True or False)

branch_id == 'B04'
False
False
False
False
True
True
True

3. Rows kept: where the mask is True

	order_id	branch_id
14060	O014061	B04
14061	O014062	B04
14062	O014063	B04

The index keeps its old labels.

The test builds the mask. The mask in square brackets after orders keeps the True rows.

Filter, count and look: 3,562 orders were sold at Jinja

```
jinja = orders[orders['branch_id'] == 'B04']
len(jinja)
```

```
3562
```

len gives the number of rows. Report it beside every filtered figure.

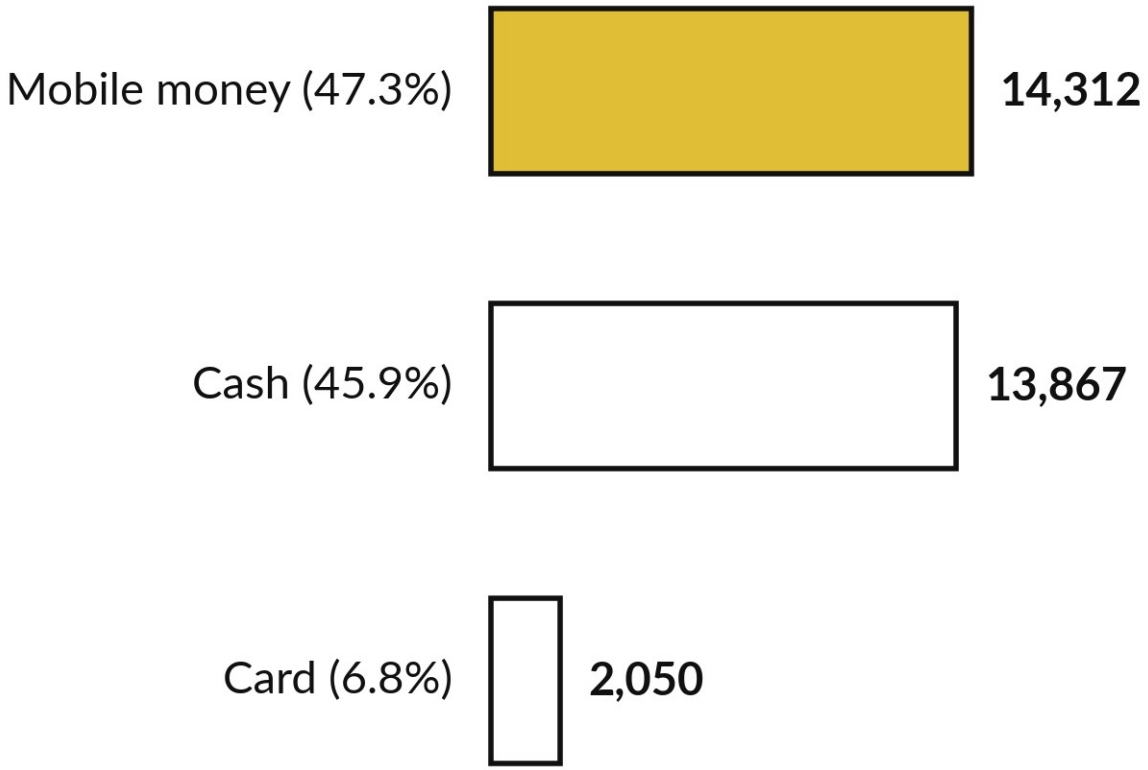
```
jinja['payment_method'].value_counts()
```

```
payment_method
Mobile money    1695
Cash            1619
Card             248
Name: count, dtype: int64
```

Mobile money leads at Jinja: 1,695 orders.

B04 is the Jinja branch. value_counts counts each different value of a Series, like a PivotTable count.

value_counts on all 30,229 orders shows Mobile money and cash nearly level



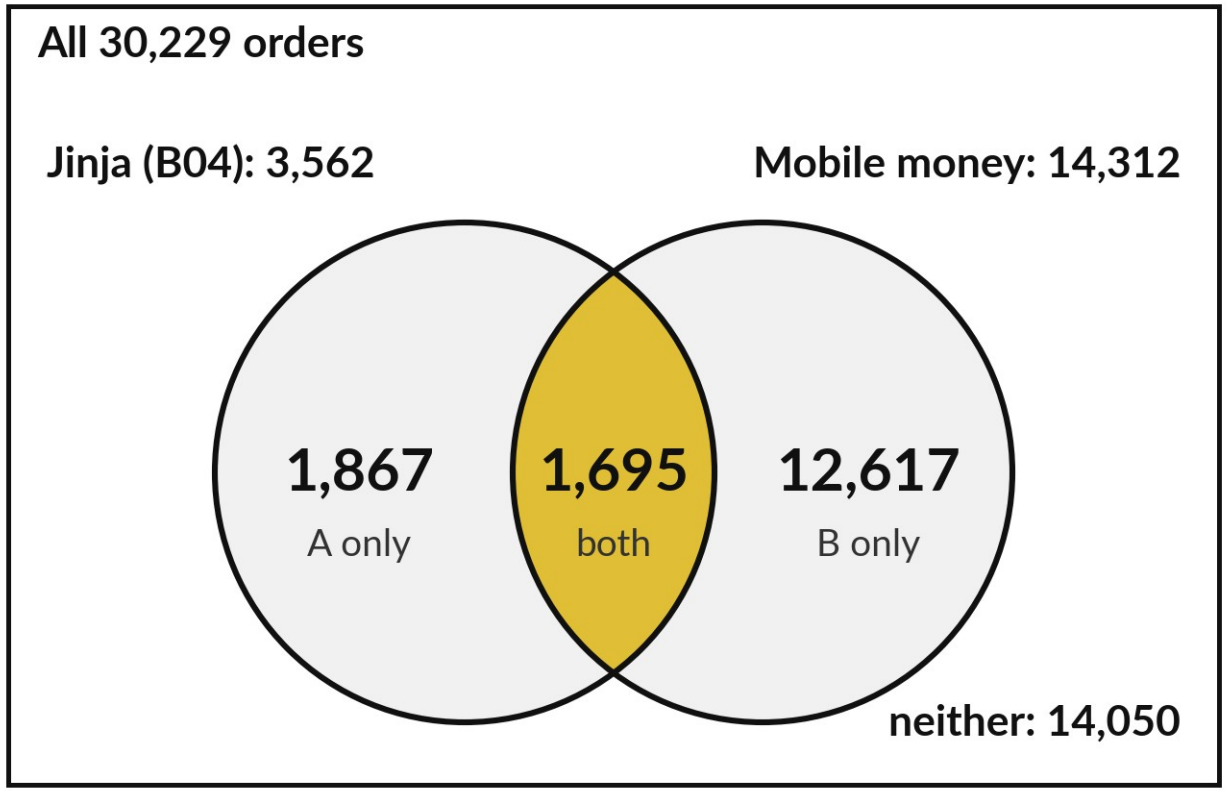
value_counts counts each different value of one variable.

It sorts the counts from largest to smallest.

value_counts(normalize=True) returns shares instead of counts.

The three counts add up to 30,229, the number of orders.

Join two conditions with & for and, | for or and ~ for not



Join two conditions

&	**and: both are True** Jinja and Mobile money	**1,695**
|	**or: at least one is True** Jinja or Mobile money	**16,179**
~	**not: flips True and False** not Mobile money	**15,917**

Real counts from the orders file. Circles are not to scale. Put each test in round brackets.

Writing and instead of &, or leaving out the brackets, fails with a confusing error

```
orders[orders['branch_id'] == 'B04' and orders['payment_method'] == 'Mobile money']
```

```
ValueError: The truth value of a Series is ambiguous.
```

```
orders[orders['branch_id'] == 'B04' & orders['payment_method'] == 'Mobile money']
```

```
TypeError: Cannot perform 'rand_' with a dtyped [object] array and
scalar of type [bool]
```

Both errors come from running these lines. Fix: join with &, each test in round brackets.

query writes the same filter as text, and both give 1,695 orders

```
mm_jinja = orders[(orders['branch_id'] == 'B04') &
                  (orders['payment_method'] == 'Mobile money')]
len(mm_jinja)
```

```
1695
```

Boolean indexing

```
mm_jinja = orders.query("branch_id == 'B04' and payment_method == 'Mobile money'")
len(mm_jinja)
```

```
1695
```

query: the condition is text. and, or, & and | all work.

Use whichever you read more easily. query is shorter when there are many conditions.

Four more filters you will use every week: isin, between, str.contains and nunique

	pandas method	Excel twin	What it does	On orders
	**isin**	OR of two tests	True where branch_id is B01 or B04	9,447 orders
	**between**	AND of two tests	True where order_value_ugx is 10,000 to 50,000, ends included	15,616 orders
	**str.contains**	SEARCH	True where payment_method holds the text Mobile	14,312 orders
	**nunique**	COUNTUNIQUE	How many different branch_id values	8 branches

The first three each build a mask, so put them inside the square brackets after orders to keep the True rows. nunique returns one number.

sort_values puts the largest orders first, and the top order is UGX 3,104,000

	order_id	order_date	branch_id	order_value_ugx
●	**O014303**	2024-02-22	B04	3,104,000
●	**O029299**	2025-05-15	B08	2,795,740
●	**O003810**	2025-04-25	B01	2,771,200
●	**O002997**	2025-01-10	B01	2,733,305
●	**O028779**	2024-12-29	B08	2,714,900

Code: orders.sort_values('order_value_ugx', ascending=False).head(5). ascending=False puts the largest first. nlargest(5) gives the same rows.

iloc counts positions and loc reads labels, and after a filter the two differ

The same rows of jinja, numbered two ways

position	label	order_id	branch_id	order_value_ugx
0	14060	O014061	B04	185865
1	14061	O014062	B04	7000
2	14062	O014063	B04	75050
3	14063	O014064	B04	3600
4	14064	O014065	B04	131100

`jinja.iloc[0:2]`

By position, counting from 0. The end is left out: positions 0 and 1, so 2 rows.

`jinja.loc[14060:14062]`

By label. The end is included: labels 14060, 14061 and 14062, so 3 rows.

The filter kept the old labels: position 0 is label 14060. Label 0 does not exist here.

loc and iloc take rows first, then columns, and give the same rows on a fresh index

```
orders.loc[0:2, ['order_id', 'branch_id']]
```

```
  order_id branch_id
0  O000001       B01
1  O000002       B01
2  O000003       B01
```

loc: rows with labels 0 to 2 (3 rows), and two named columns.

```
orders.iloc[0:3, 0:2]
```

```
  order_id  order_date
0  O000001  2024-01-06
1  O000002  2024-01-03
2  O000003  2024-01-07
```

iloc: positions 0 up to 3 (3 rows), and positions 0 and 1 for the columns.

On a fresh index both lines return the same rows. The slice ends still behave differently.

Three habits keep a filtered answer honest

A filter that runs without an error can still be wrong.



Write the row count beside the result: len(jinja) is 3,562.



Check one number a second way, for example a filter in Sheets.



Look at the mask: its sum() counts the True rows.

Where you will use this at work: load, look, filter and count

	Your job	The file you meet	What today lets you do
	**Credit analyst at a bank, with pandas**	loans.csv: 8,000 loans, 379 with no monthly income.	Filter default equal to 1 (1,004 loans) and count the blanks.
	**Health records officer at a district office**	Clinic visits: 720 district-months, 18 blank.	Keep one district (60 months) and find the blank months.
	**Sales analyst before a Power BI report**	orders.csv: 30,229 orders.	Count Jinja Mobile money orders (1,695) in one line.
	**Officer at a SACCO or cooperative, using Excel**	A monthly export of member payments.	Repeat the same filter every month by running the notebook.

Read the recipe before the lab: seven lines load, size and filter the file

```
import pandas as pd
orders = pd.read_csv('orders.csv')
print(orders.shape)
jinja = orders[orders['branch_id'] == 'B04']
print(len(jinja))
pay = jinja['payment_method'].value_counts()
print(pay.to_dict())
```

```
(30229, 7)
3562
{'Mobile money': 1695, 'Cash': 1619, 'Card': 248}
```

In plain words

Lines 1 and 2: bring in pandas, then load the file.

Line 3 prints the shape: rows, then columns.

Line 4 keeps the Jinja rows.

Line 5 prints how many: 3,562.

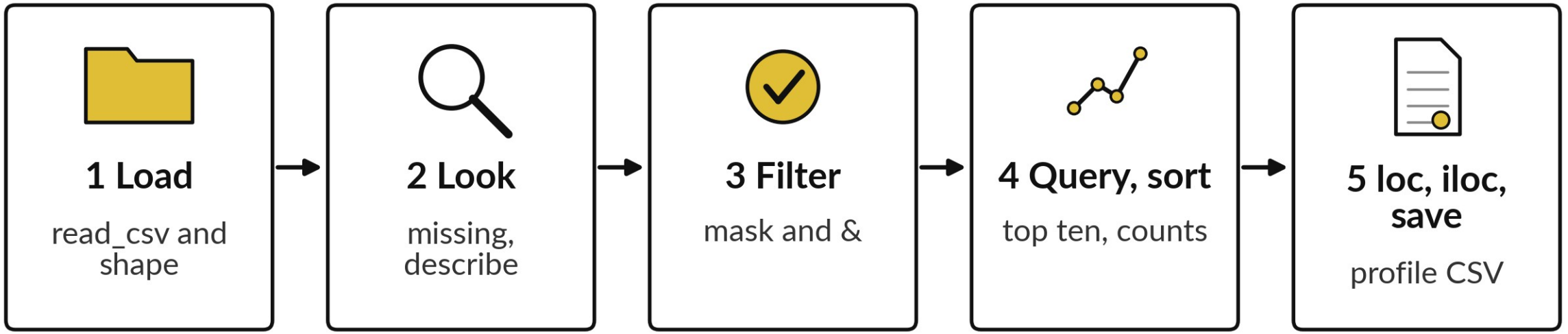
Lines 6 and 7 count the payment methods at Jinja and print them as a dictionary.



Part 2: the pandas lab

Close the slides. Open the notebook.

The lab has seven steps, and you keep a data profile at the end



Seven steps in 45 minutes, grouped in five boxes. Each step: predict, run, read, change one thing.

Check your numbers before you move on

What you compute	You should get
Shape of orders	(30,229, 7)
Missing customer_id, and its share	9,171, which is 30.3 percent
Mean and median of order_value_ugx	47,491 and 23,200
Orders at branch B04	3,562
B04 and Mobile money, and B04 or Mobile money	1,695 and 16,179
Largest order_value_ugx, and its order_id	3,104,000, O014303
Orders paid by Mobile money, Cash, Card	14,312, 13,867, 2,050
Label of the first B04 row, and its order_id	14060, O014061

If your number differs, find the step that went wrong. Do not change a number to match.

Five likely mistakes in the lab and how to fix them

What you see	What it means	What to do
ValueError: truth value of a Series is ambiguous	You wrote and or or between two tests.	Use & or |, with each test in round brackets.
TypeError mentioning rand_	You left out the round brackets around a test.	Wrap each test: (a == 1) & (b == 2).
KeyError on a column name	The name is misspelt or has the wrong case.	Print orders.columns and copy the name.
SyntaxError: cannot assign to subscript here. Maybe you meant '==' instead of '='?	You wrote one = where a test needs ==.	Use == to test and = to store.
Your count is 0	The text does not match, such as b04 for B04.	Check the value with value_counts first.

Exit check: three questions, then compare with the short answers

	Question	Short answer
	**What is the difference between a DataFrame and a Series?**	A DataFrame is a whole table. A Series is one column of it, with the same index.
	**A filter with two tests gives a TypeError that mentions rand_. What is the most likely cause?**	The two tests are not each in round brackets, so & is applied too early.
	**jinja has 3,562 rows and keeps its old labels. Does iloc with position 0 give the same row as loc with label 0?**	No. Position 0 is the first row, label 14060. Label 0 is not in jinja, so loc gives a KeyError.

Save your notebook and the orders_profile.csv file

What you keep from today

Keep your notebook and the file orders_profile.csv: a data profile with one row for each variable, giving its data type, how many values are missing and the share missing.

Our question today

How do we open a file of 30,229 rows, look at it, and ask it the first questions?

Next session

Session 8: pandas II, cleaning the mess. We clean a raw export and prove its total. Project milestone 1.

Take away

Your lab file

A data profile

The key habit

Look, then filter

Next session

Cleaning the mess